

```

"""
ACsync - Full Pipeline
=====

Loads real AC + weather data, trains a temperature-prediction model,
runs the AC coordination algorithm, and simulates power/energy/cost
savings compared to uncoordinated ("traditional") operation.

Methodology (household holdout):
    - Train on N houses.
    - Test on ONE completely unseen household (default: "Hamad").
    This checks whether the model generalizes to a household it has
    never seen, not just to a new time period in a house it already knows.

Run:
    python pipeline.py --data_dir /path/to/house/excels \
                      --hamad_file Hamad_1.xlsx \
                      --weather_file Riyadh.xlsx \
                      --out results.json
"""

import argparse
import glob
import json
import os
import pickle
import warnings
from datetime import datetime

import numpy as np
import openpyxl
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

warnings.filterwarnings("ignore")

# -----
# Config (documented assumptions - change here, not scattered in code)
# -----
COMPRESSOR_ON_THRESHOLD_KW = 0.5    # activePower above this = compressor is runni
POWER_PER_COMPRESSOR_KW = 1.5      # assumed draw of one active compressor
ELECTRICITY_RATE_SAR = 0.18        # SAR per kWh, for cost estimate
FORECAST_HORIZON_MIN = 20          # how far ahead the model predicts temperature

```

```

RESAMPLE_INTERVAL = "10min"
MAX_SIMULTANEOUS_COMPRESSORS = 2    # ACsync's safety-layer cap
FEATURES = ["temperature", "hour", "outdoor_temp", "day_of_week",
            "is_peak_hours", "compressor_on"]
TARGET = "future_temp"

# -----
# Step 1: Load one house's Excel file -> long dataframe of all AC rooms
# -----
def load_house_file(fpath, house_name, exclude_keywords=("ثلاج", "غسال", "سخان")):
    """Read every valid AC sheet in a house's Excel file into one dataframe."""
    wb = openpyxl.load_workbook(fpath, data_only=True, read_only=True)
    frames = []
    for sheet in wb.sheetnames:
        if any(k in sheet for k in exclude_keywords):
            continue
        ws = wb[sheet]
        rows = list(ws.iter_rows(values_only=True))
        if len(rows) < 2:
            continue
        header = rows[0]
        if not header or not {"activePower", "temperature", "createdAt"} <= set(h
            continue
        df = pd.DataFrame(rows[1:], columns=header)[["createdAt", "activePower",
        df["house"] = house_name
        df["room"] = sheet.strip()
        frames.append(df)
    wb.close()
    if not frames:
        return pd.DataFrame(columns=["createdAt", "activePower", "temperature", "
    return pd.concat(frames, ignore_index=True)

def load_all_houses(data_dir, exclude_files=()):
    """Load every .xlsx house file in a directory."""
    files = sorted(glob.glob(os.path.join(data_dir, "*.xlsx")))
    files = [f for f in files if os.path.basename(f) not in exclude_files]
    all_frames = []
    for fpath in files:
        house_name = os.path.basename(fpath).replace(".xlsx", "")
        all_frames.append(load_house_file(fpath, house_name))
    return pd.concat(all_frames, ignore_index=True)

# -----
# Step 2: Clean + resample to a consistent 10-minute timeline per room

```

```

# -----
def clean_and_resample(df):
    df = df.copy()
    df["createdAt"] = pd.to_datetime(df["createdAt"], errors="coerce")
    df["activePower"] = pd.to_numeric(df["activePower"], errors="coerce")
    df["temperature"] = pd.to_numeric(df["temperature"], errors="coerce")
    df = df.dropna(subset=["createdAt"])
    df["compressor_on"] = (df["activePower"] > COMPRESSOR_ON_THRESHOLD_KW).astype(int)

    out = []
    for (house, room), g in df.groupby(["house", "room"]):
        g = g.set_index("createdAt").sort_index()
        r = g[["activePower", "temperature", "compressor_on"]].resample(RESAMPLE_
            r["compressor_on"] = (r["compressor_on"] > 0.5).astype(int)
            r["house"] = house
            r["room"] = room
            out.append(r.reset_index())
    full = pd.concat(out, ignore_index=True)
    return full.dropna(subset=["temperature"])

# -----
# Step 3: Load hourly city weather and merge as outdoor_temp
# -----
def load_weather(fpath):
    wb = openpyxl.load_workbook(fpath, data_only=True)
    ws = wb["Sheet0"]
    rows = list(ws.iter_rows(min_row=2, values_only=True))
    w = pd.DataFrame(rows, columns=["temperature", "day", "time", "fullDate", "ci
    w["fullDate"] = pd.to_datetime(w["fullDate"])
    return w[["fullDate", "temperature"]].rename(
        columns={"fullDate": "createdAt", "temperature": "outdoor_temp"}
    ).sort_values("createdAt")

def merge_weather(df, weather):
    df = df.sort_values("createdAt")
    return pd.merge_asof(df, weather, on="createdAt", direction="nearest")

# -----
# Step 4: Feature engineering + forecast target
# -----
def build_features(df):
    df = df.copy()
    df["hour"] = df["createdAt"].dt.hour
    df["day_of_week"] = df["createdAt"].dt.dayofweek

```

```

df["is_peak_hours"] = df["hour"].between(13, 17).astype(int)

steps_ahead = FORECAST_HORIZON_MIN // 10
df = df.sort_values(["house", "room", "createdAt"])
grp = df.groupby(["house", "room"])
df[TARGET] = grp["temperature"].shift(-steps_ahead)
df["future_time"] = grp["createdAt"].shift(-steps_ahead)
df["gap_min"] = (df["future_time"] - df["createdAt"]).dt.total_seconds() / 60

valid = df[df["gap_min"] == FORECAST_HORIZON_MIN].dropna(
    subset=[TARGET, "outdoor_temp", "compressor_on", "temperature"]
)
return valid

# -----
# Step 5: Train models with a household-holdout split
# -----
def train_models(model_df, holdout_house):
    train = model_df[model_df["house"] != holdout_house]
    test = model_df[model_df["house"] == holdout_house]

    X_train, y_train = train[FEATURES], train[TARGET]
    X_test, y_test = test[FEATURES], test[TARGET]

    lr = LinearRegression().fit(X_train, y_train)
    lr_rmse = float(np.sqrt(mean_squared_error(y_test, lr.predict(X_test))))

    rf = RandomForestRegressor(
        n_estimators=100, max_depth=10, min_samples_leaf=10,
        random_state=42, n_jobs=-1
    ).fit(X_train, y_train)
    rf_pred = rf.predict(X_test)
    rf_rmse = float(np.sqrt(mean_squared_error(y_test, rf_pred)))

    importances = dict(sorted(
        zip(FEATURES, rf.feature_importances_.tolist()), key=lambda x: -x[1]
    ))

    test = test.copy()
    test["predicted_temp"] = rf_pred

    metrics = {
        "train_rows": int(len(train)),
        "test_rows": int(len(test)),
        "train_houses": int(train["house"].nunique()),
        "holdout_house": holdout_house,

```

```

        "linear_regression_rmse_c": round(lr_rmse, 3),
        "random_forest_rmse_c": round(rf_rmse, 3),
        "improvement_pct": round((lr_rmse - rf_rmse) / lr_rmse * 100, 1),
        "feature_importance_pct": {k: round(v * 100, 2) for k, v in importances.items()}
    }
    return rf, metrics, test

# -----
# Step 6: Coordination simulation (the "safety layer", independent of prediction)
# -----
def find_best_simulation_week(full_house_df):
    """Pick the 7-day window with the highest real simultaneous-compressor peak
    (the period that actually demonstrates the coincidence-peak problem)."""
    df = full_house_df.dropna(subset=["temperature"]).copy()
    df["date"] = df["createdAt"].dt.date
    by_time = df.groupby("createdAt")["compressor_on"].sum().rename("active").reset_index()
    by_time["date"] = by_time["createdAt"].dt.date
    daily_max = by_time.groupby("date")["active"].max()
    best_day = daily_max.idxmax()
    start = pd.Timestamp(best_day)
    return start, start + pd.Timedelta(days=7)

def simulate_coordination(house_df, start, end):
    """
    need_cooling is taken directly from the REAL recorded compressor state
    (the actual thermostat decision) – the coordinator does not invent demand,
    it only enforces the simultaneity cap on top of real, observed demand.
    """
    window = house_df[(house_df["createdAt"] >= start) & (house_df["createdAt"] < end)]
    window = window.dropna(subset=["temperature"])
    window["need_cooling"] = window["compressor_on"]

    def coordinate(group):
        g = group.sort_values("temperature", ascending=False).copy()
        active, decisions = 0, []
        for _, row in g.iterrows():
            if row["need_cooling"] and active < MAX_SIMULTANEOUS_COMPRESSORS:
                decisions.append(1)
                active += 1
            else:
                decisions.append(0)
        g["acsync_on"] = decisions
        return g

    coordinated = (

```

```

        window.groupby("createdAt", group_keys=True)
        .apply(coordinate, include_groups=False)
        .reset_index(level=0)
    )

    traditional = window.groupby("createdAt")["compressor_on"].sum()
    acsync = coordinated.groupby("createdAt")["acsync_on"].sum()

    power = pd.concat(
        [traditional * POWER_PER_COMPRESSOR_KW, acsync * POWER_PER_COMPRESSOR_KW]
    ).fillna(0)
    power.columns = ["traditional_power_kw", "acsync_power_kw"]

    interval_h = 10 / 60
    energy_trad = float((power["traditional_power_kw"] * interval_h).sum())
    energy_acsync = float((power["acsync_power_kw"] * interval_h).sum())
    peak_trad = float(power["traditional_power_kw"].max())
    peak_acsync = float(power["acsync_power_kw"].max())

    results = {
        "window_start": str(start.date()),
        "window_end": str((end - pd.Timedelta(days=1)).date()),
        "rooms": int(window["room"].nunique()),
        "timestamps": int(window["createdAt"].nunique()),
        "peak_power_kw": {"traditional": round(peak_trad, 2), "acsync": round(peak_acsync, 2)},
        "peak_reduction_pct": round((peak_trad - peak_acsync) / peak_trad * 100, 2),
        "avg_power_kw": {
            "traditional": round(float(power["traditional_power_kw"].mean()), 2),
            "acsync": round(float(power["acsync_power_kw"].mean()), 2),
        },
        "energy_kwh": {"traditional": round(energy_trad, 2), "acsync": round(energy_acsync, 2)},
        "energy_saved_kwh": round(energy_trad - energy_acsync, 2),
        "energy_reduction_pct": round((energy_trad - energy_acsync) / energy_trad * 100, 2),
        "cost_sar": {
            "traditional": round(energy_trad * ELECTRICITY_RATE_SAR, 2),
            "acsync": round(energy_acsync * ELECTRICITY_RATE_SAR, 2),
        },
        "cost_saved_sar": round((energy_trad - energy_acsync) * ELECTRICITY_RATE_SAR, 2),
        "max_simultaneous_compressors": {
            "traditional": int(traditional.max()),
            "acsync": int(acsync.max()),
        },
        "assumptions": {
            "power_per_compressor_kw": POWER_PER_COMPRESSOR_KW,
            "electricity_rate_sar_per_kwh": ELECTRICITY_RATE_SAR,
            "max_simultaneous_compressors_acsync": MAX_SIMULTANEOUS_COMPRESSORS,
            "note": "Simulated results based on recorded compressor behavior as t

```

```

        "demand baseline. Not a measured electricity-bill saving.",
    },
}
return results, power, window, coordinated

# -----
# Main
# -----
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--data_dir", required=True, help="Folder with all house .xls
    ap.add_argument("--hamad_file", required=True, help="Filename of the holdout
    ap.add_argument("--weather_file", required=True, help="City weather .xlsx (Ri
    ap.add_argument("--out", default="results.json")
    args = ap.parse_args()

    hamad_name = os.path.basename(args.hamad_file).replace(".xlsx", "")
    print(f"[1/6] Loading all houses from {args.data_dir} ...")
    raw = load_all_houses(args.data_dir)
    print(f"        -> {len(raw):,} raw rows across {raw['house'].nunique()} houses

    print("[2/6] Cleaning + resampling to 10-minute intervals ...")
    resampled = clean_and_resample(raw)

    print(f"[3/6] Loading weather from {args.weather_file} ...")
    weather = load_weather(args.weather_file)
    merged = merge_weather(resampled, weather)

    print("[4/6] Building features + forecast target ...")
    model_df = build_features(merged)
    print(f"        -> {len(model_df):,} usable training rows")

    print(f"[5/6] Training models (holdout house = {hamad_name}) ...")
    rf_model, metrics, hamad_test = train_models(model_df, hamad_name)
    print(f"        LR RMSE={metrics['linear_regression_rmse_c']} "
          f"RF RMSE={metrics['random_forest_rmse_c']} "
          f"Improvement={metrics['improvement_pct']}%")

    print("[6/6] Running coordination simulation on Hamad's house ...")
    hamad_full = merged[merged["house"] == hamad_name]
    start, end = find_best_simulation_week(hamad_full)
    sim_results, power_df, window_df, coordination_df = simulate_coordination(ham
    print(f"        Window: {sim_results['window_start']} -> {sim_results['window_e
    print(f"        Peak reduction: {sim_results['peak_reduction_pct']}% "
          f"Energy reduction: {sim_results['energy_reduction_pct']}%")

```

```

# Save model
with open("rf_model.pkl", "wb") as f:
    pickle.dump(rf_model, f)

# Build results.json for the dashboard
temp_records = window_df[["createdAt", "room", "temperature", "outdoor_temp",
if "predicted_temp" in hamad_test.columns:
    pred_map = hamad_test.set_index(["createdAt", "room"])["predicted_temp"]
    temp_records = temp_records.set_index(["createdAt", "room"])
    temp_records["predicted_temp"] = pred_map
    temp_records = temp_records.reset_index()
temp_records["createdAt"] = temp_records["createdAt"].astype(str)

coord_records = coordination_df[["createdAt", "room", "temperature", "compres
coord_records["createdAt"] = coord_records["createdAt"].astype(str)

power_records = power_df.reset_index()
power_records["createdAt"] = power_records["createdAt"].astype(str)

output = {
    "generated_at": datetime.now().isoformat(),
    "household": hamad_name,
    "model_metrics": metrics,
    "simulation": sim_results,
    "temperature_data": temp_records.to_dict(orient="records"),
    "power_data": power_records.to_dict(orient="records"),
}

with open(args.out, "w", encoding="utf-8") as f:
    json.dump(output, f, ensure_ascii=False, indent=2)
print(f"\nSaved: {args.out}")

if __name__ == "__main__":
    main()

```